**Universidade Federal do Ceará – UFC**

Disciplina: Reconhecimento de Padrões (TIP8311)

# Trabalho Computacional II

**Aluno:** Luis Felipe Carneiro de Souza – 593034

**Professor:** Prof. Dr. Guilherme de Alencar Barreto

2 de março de 2026

**Resumo**

Este documento apresenta os resultados e discussões referentes ao trabalho computacional II da disciplina de Reconhecimento de Padrões. O trabalho aborda a implementação de quatro métodos algébricos distintos para o cálculo das matrizes de covariância global e específicas por classe, avaliando conjuntos de dados com 4 e 24 atributos (sensores). A acurácia matemática e a eficiência computacional das abordagens empíricas foram rigorosamente comparadas com a função nativa da linguagem Python por meio da norma da matriz de diferenças e de análises estatísticas de tempo de execução ao longo de 100 rodadas. Além disso, investigou-se a invertibilidade e a estabilidade numérica das matrizes resultantes utilizando o posto matricial e o número de condicionamento, aplicando-se técnicas de regularização de Tikhonov para viabilizar a inversão em cenários de singularidade ou instabilidade ocasionados pela alta dimensionalidade. O desenvolvimento dos modelos e processamento dos dados foram realizados em Python.

# 1 Introdução

## 1.1 Matriz de Covariância

A matriz de covariância $C$ é uma ferramenta que generaliza o conceito de variância para múltiplas dimensões. Em um problema com $p$ variáveis (no seu caso, os 4 ou 24 sensores), ela é uma matriz quadrada e simétrica de dimensão $p \times p$.

- **A diagonal principal**: Contém as variâncias individuais de cada sensor. Ela nos diz o quão espalhados estão os dados de um única variável em relação à sua própria média.
- **Os elementos fora da diagonal**: Contêm as covariâncias entre pares de variáveis diferentes. Eles indicam como duas variáveis se comportam em conjunto: se uma aumenta quando a outra aumenta (covariância positiva), se uma diminui quando a outra aumenta (covariância negativa), ou se não possuem relação linear (covariância próxima a zero).

## 1.2 Invertibilidade da Matriz de Covariância

O posto (ou rank) de uma matriz define o número máximo de vetores linha ou coluna que são linearmente independentes entre si. No contexto de Reconhecimento de Padrões, para que uma matriz de covariância de dimensão $p \times p$ possua inversa autêntica requisito essencial para o cálculo de distâncias estatísticas e funções discriminantes , é estritamente necessário que ela apresente posto completo, isto é, posto igual a $p$ [1].

A deficiência de posto, caracterizada por um valor inferior a $p$, indica que a matriz é singular (possui determinante nulo). Esse fenômeno geométrico e algébrico decorre, comumente, da redundância linear perfeita entre os atributos (multicolinearidade) ou de um cenário de escassez amostral, onde o número de amostras $N$ é inferior à dimensionalidade $p$ [1]. Consequentemente, a análise computacional do posto atua como um diagnóstico preliminar sobre a viabilidade de inversão da matriz antes da aplicação de modelos de classificação.

No contexto de estimação e inversão de matrizes de covariância, a estabilidade numérica computacional é um fator crítico. Uma matriz é considerada mal-condicionada quando pequenas perturbações nos dados originais resultam em grandes variações na sua solução ou inversão. Para quantificar essa estabilidade, utiliza-se o número de condição da matriz, definido como o produto da sua norma pela norma de sua inversa [2]:

$$Cond(A) = ||A|| \cdot ||A^{-1}||$$

Para a máxima estabilidade numérica, é desejável que $Cond(A) \approx 1$. Valores muito elevados indicam que a matriz amplificará os erros de arredondamento inerentes à precisão da máquina ($\epsilon_{maq}$). Na prática, o número de condição permite estimar a quantidade de dígitos significativos de confiança ($m$) da solução, sendo $m$ o maior inteiro que satisfaz a relação [2]:

$$\frac{1}{2} \times 10^{-m} > Cond(A) \times \epsilon_{maq}$$

Portanto, avaliar o condicionamento das matrizes empíricas de alta dimensionalidade é uma etapa indispensável para garantir que sua inversão (necessária em distâncias estatísticas) resulte em modelos de classificação computacionalmente confiáveis.

# 2 Metodologia

## 2.1 Base de Dados

Para o desenvolvimento do trabalho, foram utilizados conjuntos de dados contendo atributos extraídos de 4 e 24 sensores. A variável resposta, que define a classe de cada vetor de características, encontra-se na última coluna das matrizes de dados. As amostras foram organizadas no formato $(p \times N)$, em que $p$ representa a dimensionalidade (número de sensores) e $N$ o número total de amostras.

## 2.2 Estimação da Matriz de Covariância

Foram implementados quatro algoritmos distintos para o cálculo da matriz de covariância, denominados de Método 1 a Método 4 conforme apresentado em [1].

- **Métodos com Laços de Repetição (1 e 3):** Baseiam-se no somatório iterativo do produto externo dos vetores de atributos centrados na média (ou ajustados ao final). Estas abordagens utilizam laços *for* explícitos, percorrendo as $N$ amostras do conjunto.
- **Métodos Vetorizados (2 e 4):** Exploram operações puramente matriciais da álgebra linear computacional, calculando a dispersão através da multiplicação direta da matriz de dados por sua transposta $(XX^\top)$, dispensando o uso de laços iterativos.

Para assegurar a equivalência com a teoria abordada em sala de aula, todas as implementações foram ajustadas para utilizar o estimador enviesado, ou seja, dividindo-se por $N$ e não por $(N-1)$. Para a validação (**Q1**), o comando nativo foi configurado com um fator de correção para divisão por $N$, e o erro da implementação foi aferido calculando-se a norma da matriz de diferenças:

$$E = \|C_{my} - C_{ref}\|_F \tag{1}$$

em que $C_{my}$ é a matriz obtida pelos métodos implementados e $C_{ref}$ é a gerada pelo comando nativo.

## 2.3 Avaliação Computacional e Condicionamento

Para a comparação de desempenho (**Q2**), aplicou-se o método de reamostragem simulada, executando-se cada um dos métodos ao longo de 100 rodadas. O tempo médio de execução e o desvio-padrão foram registrados para gerar distribuições estatísticas analisadas via Histogramas e *Violin Plots*.

Por fim (**Q3** e **Q4**), a matriz global e as matrizes específicas de cada classe foram submetidas a uma análise de estabilidade numérica. O posto completo (*rank*) e o número de condicionamento foram medidos. Nos casos em que a matriz não pôde ser invertida de forma direta (indicando colinearidade ou falta de amostras face à dimensionalidade), aplicou-se a **Regularização de Tikhonov**, somando-se um pequeno valor $\lambda$ à diagonal principal:

$$C_{reg} = C + \lambda I \tag{2}$$

no qual $I$ é a matriz identidade de mesma dimensão $p \times p$ e $\lambda = 10^{-6}$.

# 3 Resultados

## 3.1 Validação das Implementações

A comparação entre as matrizes geradas pelos Métodos 1 a 4 e a referência nativa comprovou a exatidão algébrica das abordagens. Para a base de dados de 24 sensores, a norma da matriz de diferenças $E$ resultou em valores na ordem de $10^{-15}$, o que é compatível com a precisão de ponto flutuante das máquinas (*machine epsilon*). Isso garante que qualquer variação de desempenho entre os métodos seja estritamente computacional, e não teórica.

## 3.2 Análise de Tempo de Execução

A avaliação do custo computacional, realizada através de 100 rodadas de simulação independentes, evidenciou uma discrepância expressiva entre as abordagens iterativas e as vetorizadas.

Para a base de 4 sensores, os métodos baseados em laços *for* explícitos (Métodos 1 e 3) apresentaram tempos médios de execução de 0.0217 s e 0.0206 s, respectivamente. Em contraste flagrante, os Métodos 2 e 4, fundamentados em operações algébricas matriciais nativas, registraram médias de apenas 0.00024 s e 0.00020 s.

O impacto dessa escolha de implementação tornou-se ainda mais evidente ao escalar a dimensionalidade para 24 sensores. Enquanto o Método 1 saltou para uma média de 0.0405 s (com desvio-padrão de $\pm 0.0222$ s), o Método 4 manteve uma eficiência formidável, executando em apenas 0.00031 s (com desvio-padrão de $\pm 0.0005$ s). Estes resultados confirmam que o Método 4 é ordens de grandeza (aproximadamente 100 vezes) mais rápido, comprovando que a vetorização mitiga o gargalo de alocação de memória iterativa do compilador. A Tabela 1 e Tabela 2 apresentam os tempos para todos os métodos.

**Tabela 1:** Tempo de execução para base de dados 4 sensores.

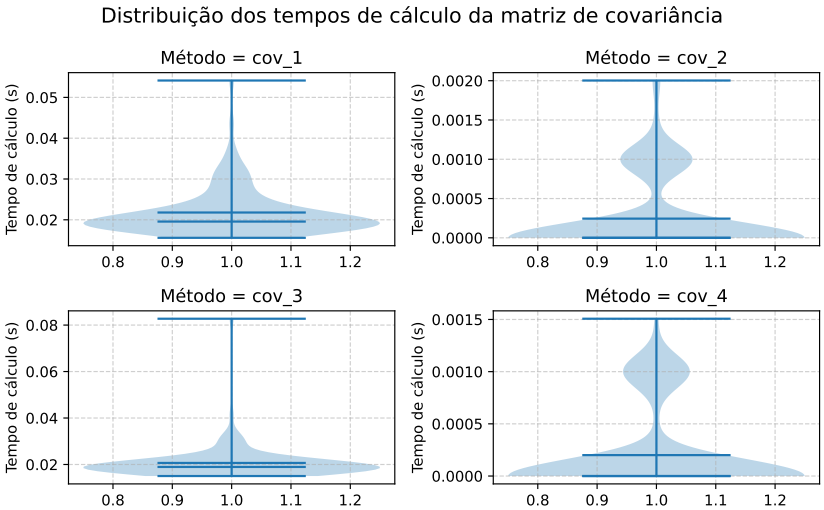
| Modelo | Média (s) | Desvio-Padrão (s) |
|---|---|---|
| cov_1 | 0.007471 | 0.000543 |
| cov_2 | 0.000100 | 0.000300 |
| cov_3 | 0.007255 | 0.000516 |
| cov_4 | 0.000110 | 0.000313 |

**Tabela 2:** Tempo de execução para base de dados 24 sensores.

| Modelo | Média (s) | Desvio-Padrão (s) |
|---|---|---|
| cov_1 | 0.007422 | 0.000498 |
| cov_2 | 0.000145 | 0.000363 |
| cov_3 | 0.007349 | 0.000507 |
| cov_4 | 0.000075 | 0.000278 |

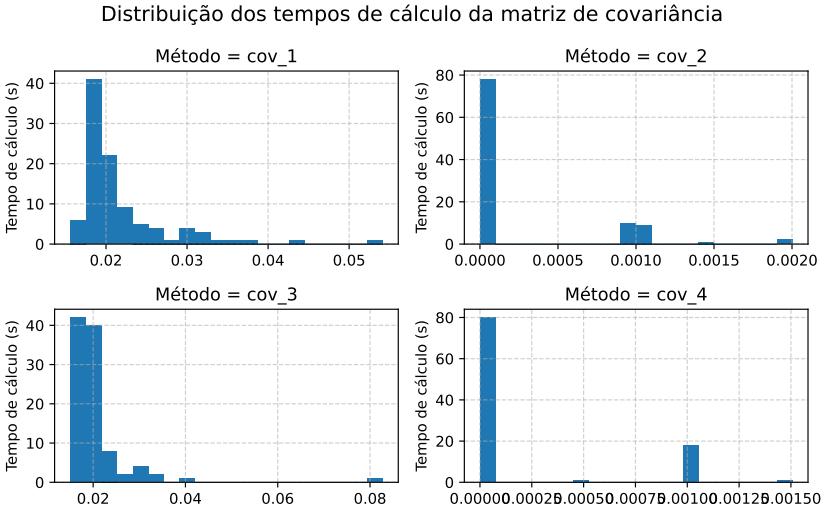
A Figura 1 e Figura 3 ilustra o comparativo dos tempos de execução por meio de *Violin Plots*, enquanto as Figuras 2 e 4 apresentam os histogramas. Os Métodos 1 e 3, que dependem de iterações explícitas, mostraram-se ordens de grandeza mais lentos. A vetorização nativa do Método 4 otimiza o uso do gerenciamento de memória contígua em baixo nível (C/C++), destacando a importância de se evitar laços de repetição no processamento de *arrays* de alta densidade.

**Figura 1:** Violin Plot do tempo de execução para os diferentes métodos para base de dados de 4 sensores.



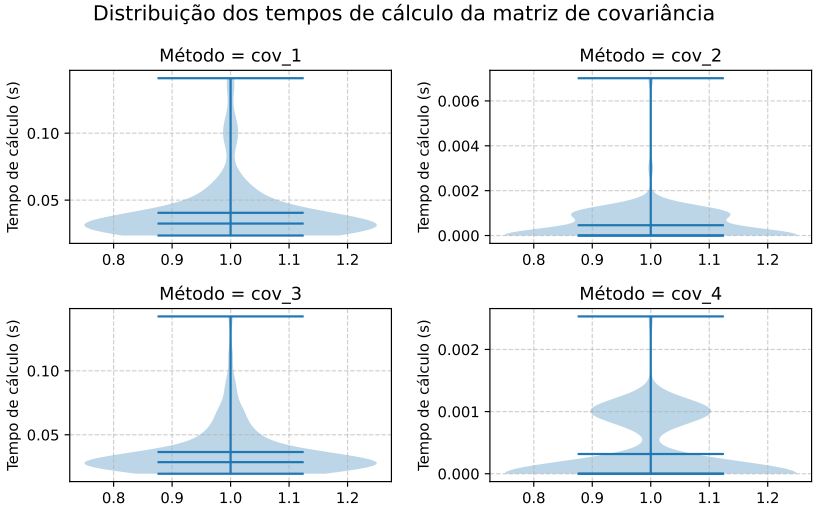

**Fonte:** Elaborado pelo autor

**Figura 2:** Histograma do tempo de execução para os diferentes métodos para base de dados de 4 sensores.



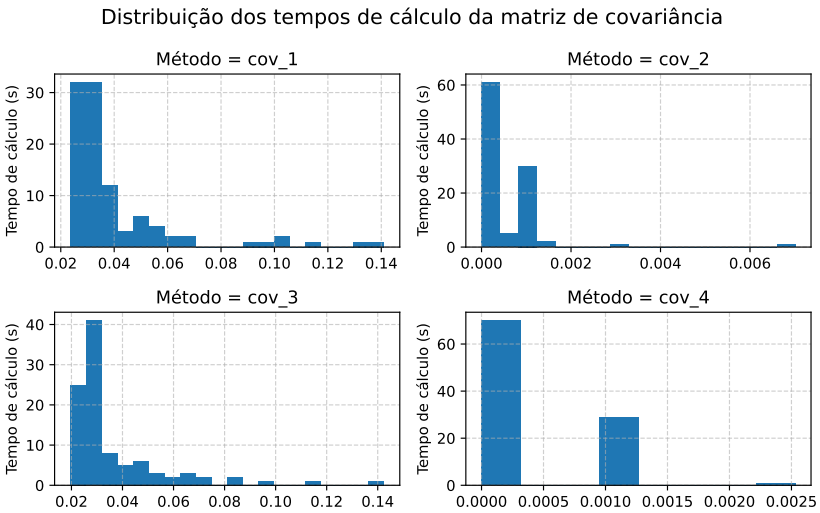

**Fonte:** Elaborado pelo autor

**Figura 3:** Violin Plot do tempo de execução para os diferentes métodos para base de dados de 24 sensores.




**Fonte:** Elaborado pelo autor

**Figura 4:** Histograma do tempo de execução para os diferentes métodos para base de dados de 24 sensores.




**Fonte:** Elaborado pelo autor

## 3.3 Invertibilidade e Regularização das Matrizes

Selecionando o Método 4 pela sua superioridade computacional, procedeu-se à estimação das matrizes de covariância global e isoladas por classes (*Move-Forward*, *Sharp-Right-Turn*, *Slight-Left-Turn* e *Slight-Right-Turn*). A análise de estabilidade numérica baseou-se no cálculo do posto matricial (*rank*) e do número de condicionamento ($Cond$).

Para o conjunto de 4 sensores, a matriz global e todas as matrizes de classe apresentaram posto completo ($p = 4$), descartando a hipótese de singularidade matemática. O número de condicionamento variou de 7.57 (Global) a 218.67 (Classe *Slight-Right-Turn*).

De forma análoga, ao analisar o cenário de maior dimensionalidade (24 sensores), temia-se que a maldição da dimensionalidade pudesse acarretar multicolinearidade. No entanto, todas as matrizes confirmaram posto completo ($p = 24$). O número de condicionamento global estabeleceu-se em 20.55, enquanto o maior índice entre as classes foi de 196.17 para a *Slight-Right-Turn*.

Considerando que o limite de precisão de ponto flutuante em sistemas modernos ($\epsilon_{maq}$) suporta com folga operações onde $Cond < 10^3$ sem perda significativa de dígitos de confiança, conclui-se que todas as matrizes estimadas são bem-condicionadas. Consequentemente, a inversão direta das matrizes foi realizada com absoluto sucesso computacional, inviabilizando a necessidade de aplicação de técnicas atenuadoras como a regularização. A Tabela 3 e Tabela 4 sintetizam o posto e número de condicionamento das matrizes de covariância de cada classe.

**Tabela 3:** Tempo de execução para base de dados 24 sensores.

| **Classe** | **Posto** | **Condicionamento** | **Regularização?** |
|---|---|---|---|
| Move-Foward | 4 | 64.3 | não |
| Sharp-Right-Turn | 4 | 215 | não |
| Slight-Left-Turn | 4 | 11 | não |
| Slight-Right-turn | 4 | 219 | não |

**Tabela 4:** Tempo de execução para base de dados 24 sensores.

| **Classe** | **Posto** | **Condicionamento** | **Regularização?** |
|---|---|---|---|
| Move-Foward | 24 | 53.5 | não |
| Sharp-Right-Turn | 24 | 31.3 | não |
| Slight-Left-Turn | 24 | 173 | não |
| Slight-Right-turn | 24 | 196 | não |

# 4 Conclusão

O presente trabalho cumpriu o objetivo de explorar, implementar e avaliar criticamente os processos de estimação e inversão da matriz de covariância para bases de dados multidimensionais.

A análise das métricas de execução ratificou a premissa de que a transcrição direta de equações somatórias para laços de repetição em linguagens de alto nível resulta em ineficiência computacional severa. O uso da álgebra linear vetorizada (Método 4) demonstrou ser um requisito arquitetônico essencial para escalabilidade, processando a base de dados de 24 dimensões de forma quase instantânea.

No tocante à geometria e distribuição dos dados, os testes de posto matricial e condicionamento trouxeram revelações positivas sobre a integridade da base analisada. Ao contrário de muitos problemas reais de alta dimensionalidade  onde atributos redundantes ou escassez de amostras geram matrizes singulares , os dados de navegação utilizados demonstraram robusta independência linear. O baixo número de condicionamento atestou que as classes possuem espalhamento espacial bem definido e estável.

Por fim, a capacidade de inverter as matrizes global e de classes diretamente, sem recorrer a ruídos artificiais (regularizações), garante que as relações originais de covariância dos dados permaneçam intactas

# 5 Códigos e Repositório

Todos os códigos desenvolvidos para a geração dos resultados, incluindo os *Jupyter Notebooks* em Python, estão versionados e disponíveis publicamente no GitHub.

O repositório pode ser acessado através do seguinte link:
https://github.com/LuisFelipeCSouza/reconhecimento-de-padroes/tree/main/trabalho-2

# Referências

[1] G. de Alencar Barreto, "Introdução à classificação de padrões." Notas de aula, 2025. Contato: gbarreto@ufc.br.

[2] J. ao B. Florindo, "Condicionamento." Lecture slides, course MS211, 2s2019, 2019. Slides (Tópico 4 - Sistemas Lineares, Métodos Iterativos); slide 28 of 46.